Course Project Report

Classical AI Planning for Disaster-Response Logistics

Team Details

Sl. No.	Name of Student	Roll Number	Department
1	Rupesh Kumar Singh	21ME31037	Mechanical engineering
2	Pawan Kumar	21EC37029	Electronics and electrical communication engineering
3	Dheeraj Kumar Prajapati	21ME31021	Mechanical engineering

1 Problem Formulation

1.1 Problem Statement

This project addresses disaster-response logistics planning in a city where part of the road network is blocked. The planning system must coordinate resource movement, road-clearing operations, and medical treatment actions to ensure that all required victim locations are served.

The domain is modeled as a symbolic planning problem using state facts and action operators. A state is represented by facts such as road connectivity and status (`connected`, `clear`, `blocked`), resource positions (`at`), and victim status (`victims_untreated`, `victims_treated`). The objective is to compute a valid action sequence that transforms the initial state into one that satisfies all goal conditions.

1.2 Importance and Motivation

In disaster scenarios, delayed coordination between clearance and treatment teams can significantly increase risk to affected populations. An AI planner can support decision-making by generating transparent, step-by-step operational plans under resource and infrastructure constraints. This problem is both practically relevant and academically suitable for classical AI methods.

1.3 Interdisciplinary Significance

The project combines:

- disaster-management reasoning (road accessibility, team deployment, victim servicing), and
- classical AI planning (state-space search, explicit preconditions/effects, heuristic guidance).

This integration is meaningful because it converts domain constraints into formal computational structures that are interpretable and verifiable.

1.4 Inputs, Outputs, and Decision Process

Inputs

- Locations and road topology.
- Road status for each link (clear or blocked).
- Initial locations of available resources.
- Untreated victim locations and required treated goal locations.
- Search algorithm and heuristic configuration.

Outputs

- A feasible plan (ordered action sequence), or failure if no valid plan exists.
- Performance metrics: runtime, nodes expanded, and plan length.

Decision Process

- Start from the initial symbolic state.
- Repeatedly select and expand frontier states according to the selected search strategy.
- Apply only actions whose preconditions are satisfied.
- Stop when goal facts are reached or frontier is exhausted.

1.5 Problem Characterization

- **Deterministic:** action effects are fixed.
- **Static during planning:** no exogenous changes occur while planning.
- **Discrete:** both states and actions are symbolic/discrete.
- **Constrained:** action preconditions enforce connectivity, road condition, and resource presence.
- **Goal-oriented:** objective is to satisfy all treatment goals with a valid plan.

2 Method and Justification

2.1 Why the Chosen Method is Appropriate

Classical graph-search planning is suitable because the domain is naturally represented with symbolic states and deterministic transitions. Explicit preconditions and effects allow precise feasibility checks for each action. The selected search family supports both uninformed baselines (BFS, UCS) and informed variants (GBFS, A*) for efficiency analysis.

The method is appropriate with respect to:

- **Feasibility:** domain rules are straightforward to encode with action operators.
- **Interpretability:** generated plans are readable and traceable.
- **Efficiency:** heuristic-guided methods can significantly reduce node expansion on complex scenarios.

2.2 Algorithm Design and Methodology

The planning pipeline is:

1. Load a scenario (built-in simple/complex or user-provided custom JSON).
2. Construct initial facts, goal conditions, and action set.
3. Initialize the frontier and best-cost memory.
4. Select next state using algorithm-specific priority.
5. Check goal satisfaction.
6. Expand applicable actions and generate successor states.
7. Update frontier only if a better state cost is discovered.
8. Return success (plan + metrics) or failure.

2.3 Algorithm 1: Multi-Algorithm Disaster-Response Planner

Require: initial state s_0, goal facts G, action set A, algorithm `alg`, heuristic h (optional)
Ensure: valid plan P and performance metrics, or failure

1. Initialize timer, frontier, tie-breaker counter, and `best_cost`.
2. Push s_0 with empty plan and zero path cost.
3. While frontier is not empty:
4. Pop state with lowest score by selected algorithm.

5. Skip if popped path cost is worse than recorded best for that state.
6. If state satisfies G, return success with plan and metrics.
7. Increment expanded-node count.
8. For each action in A:
9. If action is applicable, execute to get successor state.
10. Compute successor cost (`current_cost + 1`).
11. If successor improves best known cost:
12. Record cost and push successor with score:
    - BFS: plan length
    - UCS: path cost
    - GBFS: heuristic value
    - A*: path cost + heuristic value
13. If frontier becomes empty without reaching G, return failure.

3 Implementation Details

3.1 System Design

The software is organized into modular components with clear responsibilities:

- `domain_models.py` defines core immutable state and action models.
- `disaster_scenario.py` and `complex_scenario.py` define scenario facts and generate domain actions.
- `planner.py` implements scenario loading, search execution, CLI orchestration, and result reporting.
- `heuristics.py` provides heuristic functions and registry.
- `benchmarks.py` runs repeated experiments and exports results.
- `visualization.py` renders state transitions on a road-network graph.

This structure minimizes coupling while supporting extensibility (new scenarios, heuristics, and algorithms).

3.2 Important Modules and Functions

- `run_planner(...)` in `planner.py`: core search loop and metrics reporting.
- `load_custom_scenario(...)` in `planner.py`: JSON validation and conversion to planner data.
- `generate_domain_actions(...)` in `disaster_scenario.py`: creates `Move`, `Clear`, and `Treat` operators.
- `get_heuristic(...)` in `heuristics.py`: resolves selected heuristic by name.
- `run_benchmarks(...)` in `benchmarks.py`: quantitative comparison across scenarios and algorithms.

3.3 Key Data Structures

- **State:** immutable fact set (`frozenset[tuple[str, ...]]`) for safe hashing.
- **Action:** symbolic operator with preconditions and add/delete effects.
- **Frontier:** priority queue containing score, tie-break value, cost, state, and current plan.
- **Best-cost map:** tracks lowest discovered path cost for each visited state.

3.4 GitHub Repository

The repository includes source code, test suite, documentation, and usage instructions.

Repository link: `https://github.com/rupeshksingh/AIFA-Project`

4 Results and Performance Analysis

4.1 Experimental Setup

- **Scenarios:** built-in `simple` and `complex`.
- **Algorithms compared:** BFS, UCS, GBFS, A*.
- **Heuristic for informed search:** `untreated_victims`.
- **Repetitions:** 5 runs per scenario-algorithm pair.
- **Software environment:** Python 3.x CLI.
- **Hardware/OS:** AMD Ryzen 5 5500U with Radeon Graphics, 8 GB RAM, 64-bit operating system.
- **Metrics:** success rate, mean time, mean nodes expanded, mean plan length.

4.2 Quantitative Results

Scenario	Algorithm	Heuristic	Runs	Success Rate	Mean Time (s)	Mean Nodes Expanded
simple	bfs	none	5	1.00	0.0001	13.00
simple	ucs	none	5	1.00	0.0001	13.00
simple	gbfs	untreated_victims	5	1.00	0.0001	11.00
simple	astar	untreated_victims	5	1.00	0.0001	11.00
complex	bfs	none	5	1.00	0.0144	785.00
complex	ucs	none	5	1.00	0.0143	785.00
complex	gbfs	untreated_victims	5	1.00	0.0023	109.00
complex	astar	untreated_victims	5	1.00	0.0112	541.00

4.3 Discussion

All algorithms achieved full success on sampled runs for both scenarios. In the complex scenario, GBFS produced the strongest runtime and expansion performance among evaluated methods, indicating that heuristic guidance aligned well with domain structure. A* reduced node expansion compared with uninformed search but remained slower than GBFS due to the additional path-cost balancing term. Plan lengths were identical in sampled settings, suggesting that scenario constraints strongly determine minimum required action count.

4.4 Visualizations and Benchmark Artifacts

The following visualizations were generated from benchmark exports and included as performance evidence.

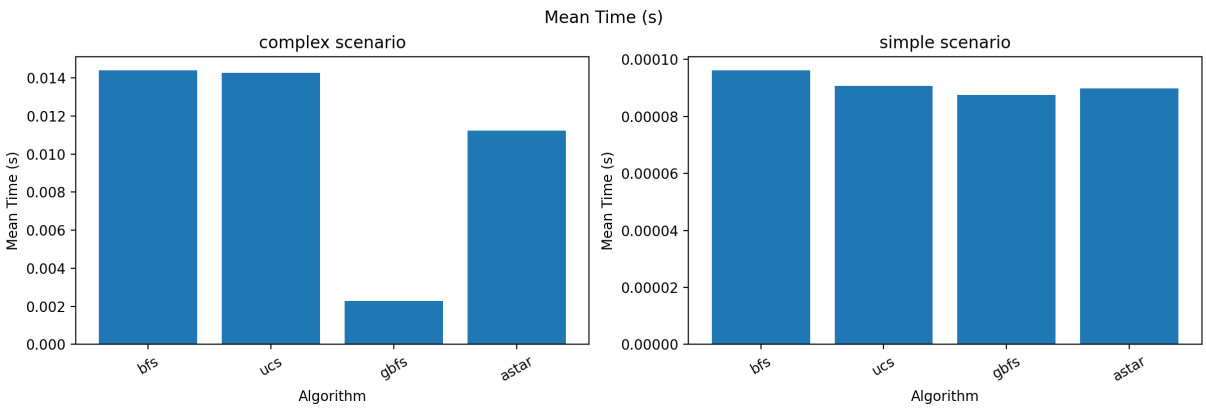

Figure 1: Mean runtime across algorithms for simple and complex scenarios.

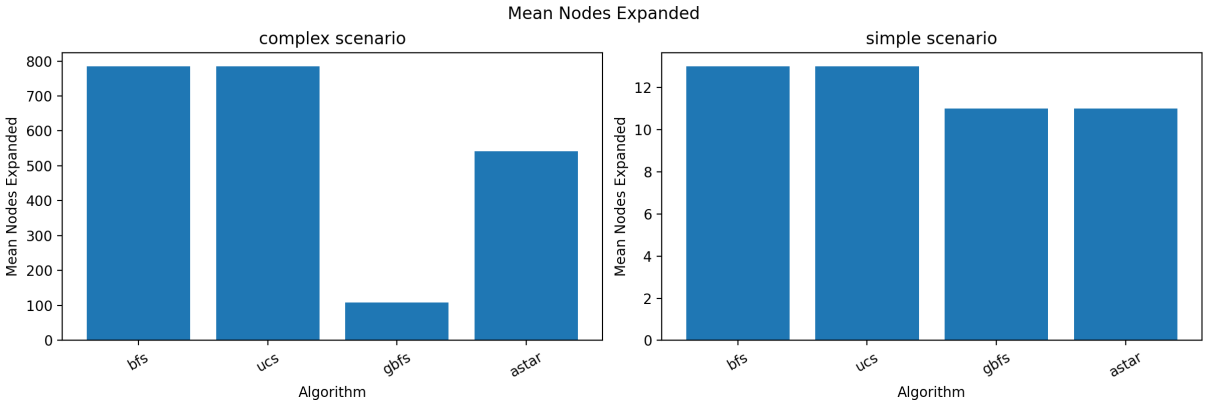

Figure 2: Mean nodes expanded across algorithms for simple and complex scenarios.

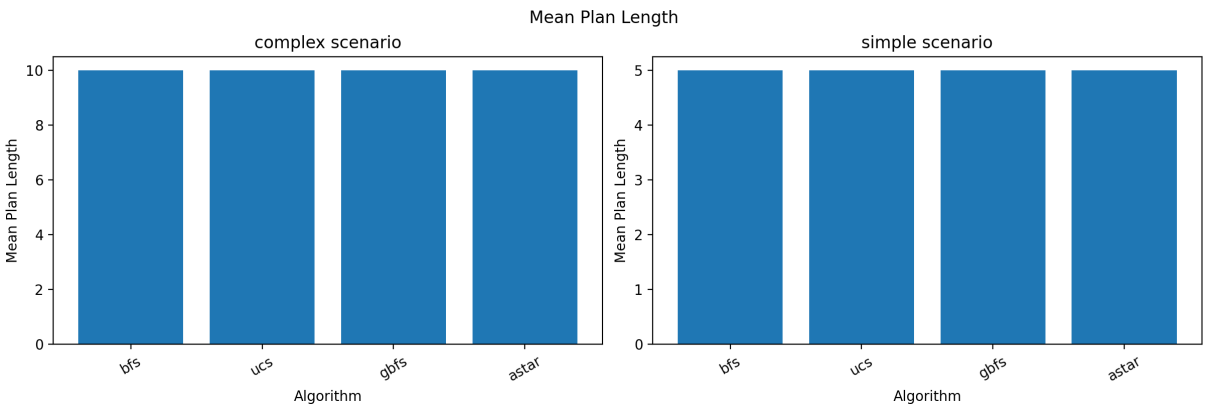

Figure 3: Mean plan length across algorithms for simple and complex scenarios.

Benchmark source artifacts:

- `benchmark_results.csv`
- `benchmark_results.json`

These artifacts preserve reproducibility and provide the raw values used in both tables and figures.

4.5 Path Execution Visualizations

In addition to aggregate benchmark charts, path-level visualizations were generated from planner execution traces to show state transitions from initial state to goal state.

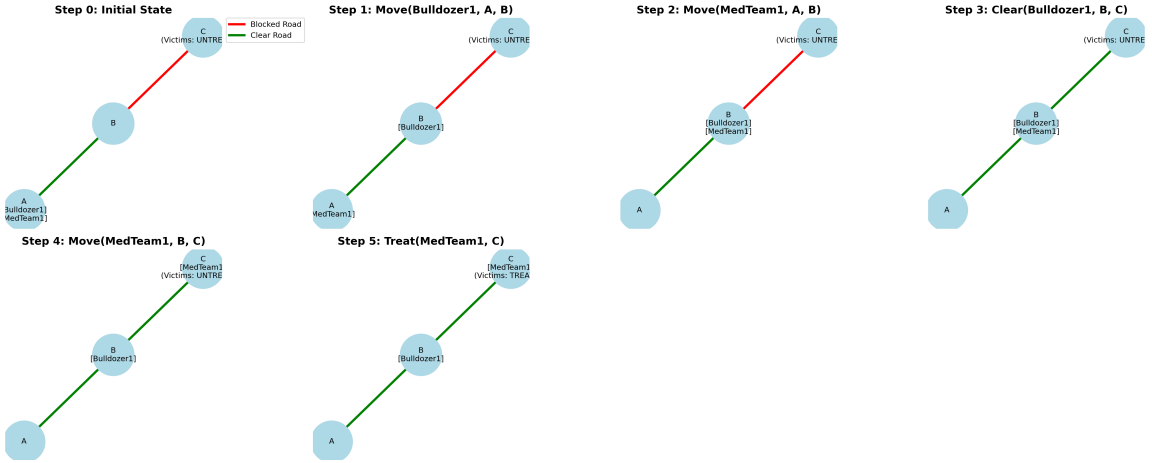

Figure 4: Step-by-step path execution summary for the simple scenario using A* with the untreated_victims heuristic.

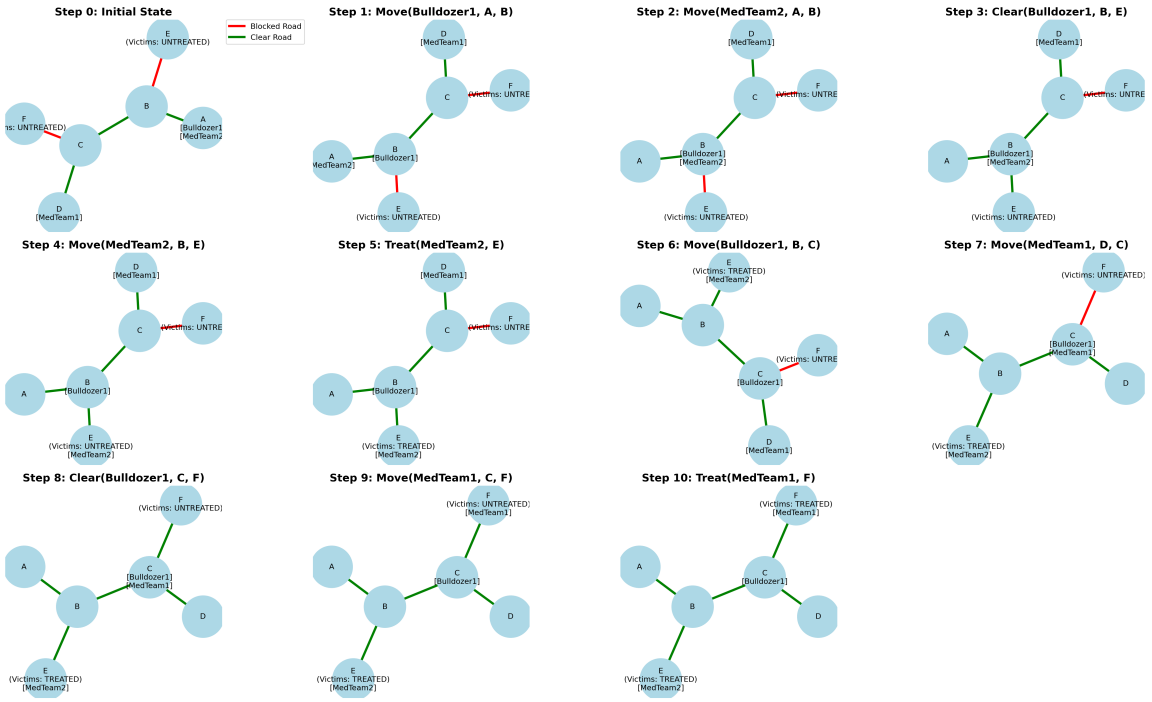

Figure 5: Step-by-step path execution summary for the complex scenario using A* with the untreated_victims heuristic.

References to Implementation Artifacts

- Primary planner: `planner.py`
- Domain model: `domain_models.py`
- Scenario definitions: `disaster_scenario.py`, `complex_scenario.py`

- Heuristics: `heuristics.py`
- Benchmark runner: `benchmarks.py`
- Visualization module: `visualization.py`
- Project overview and run instructions: `README.md`